

Harbin Institute of Technology (Shenzhen) Project Report

Image Processing (COMP5033)
2025–2026 Spring Semester
Lecturer: Prof. Weizheng Zhang

Project No.:	1	Student Name:	[YOUR NAME]
Student ID:	[YOUR STUDENT ID]	Date of Submission:	2026-05-30

1 Project Content

This project addresses image enhancement and restoration for three 224×224 grayscale images, each corrupted by a different noise type. The objectives are: (1) identify the noise model through visual and statistical analysis, and (2) apply appropriate denoising filters implemented entirely from scratch using NumPy. Only cv2 I/O functions and NumPy arithmetic are used; no library denoising functions are called.

2 Method Description

2.1 Image 1 – Salt Noise → 5×5 Median Filter

Noise analysis: Image 1 (a van on cobblestones) contains isolated, anomalously bright pixels: ~3,353 pixels (6.7%) have intensity > 250, appearing as discrete white specks with no spatial correlation. This is classic *salt noise* (impulse noise).

Method – 5×5 Median filter: The median is an order-statistic estimator robust to outliers; it replaces each pixel with the neighbourhood median, eliminating isolated bright impulses without blurring edges. A 5×5 kernel handles clusters of 2–3 adjacent noise pixels. **Implementation:** reflect-pad image by 2, extract 5×5 windows via *sliding_window_view*, then compute *np.median* across the window axes.

2.2 Image 2 – Gaussian Noise → 7×7 Gaussian Filter ($\sigma=2.0$)

Noise analysis: Image 2 (a dog) shows uniformly distributed fine-grained intensity variations across all regions ($\sigma \approx 41.6$ DN), with no spatial pattern or directionality—consistent with additive zero-mean *Gaussian noise* from thermal sensor effects.

Method – 7×7 Gaussian filter ($\sigma=2.0$): For zero-mean Gaussian noise, linear averaging reduces noise variance by a factor proportional to kernel area. The Gaussian weighting minimises the spatial-frequency bandwidth product, limiting edge blurring. A larger kernel is chosen given the high noise level. **Implementation:** $K(x,y) = \exp(-(x^2+y^2)/(2\sigma^2))/\Sigma K$; convolve via *sliding_window_view*: $\text{output}(i,j) = \Sigma K(x,y) \cdot I(i+x,j+y)$.

2.3 Image 3 – Mixed Noise → Median (3×3) + Freq-domain LPF

Noise analysis: Image 3 (an "ABUNDANCE" sign) shows both isolated impulse pixels (~1,682) and a quasi-periodic dot pattern visible as concentric spectral rings in the Fourier domain, indicating *mixed salt-and-pepper + periodic noise*.

Method – Two-stage approach:

Stage 1 (3×3 Median): removes isolated impulse pixels before frequency analysis.

Stage 2 (Frequency-domain Gaussian LPF): $F = \text{FFT2}(I)$; centre the spectrum; multiply by Gaussian mask $H(u,v) = \exp(-D^2/(2D_{\text{max}}^2))$ with $D_{\text{max}} = 0.25 \cdot \min(H,W)$; recover via inverse FFT. The Gaussian roll-off avoids ringing artefacts.

3 Experiment Results and Analysis

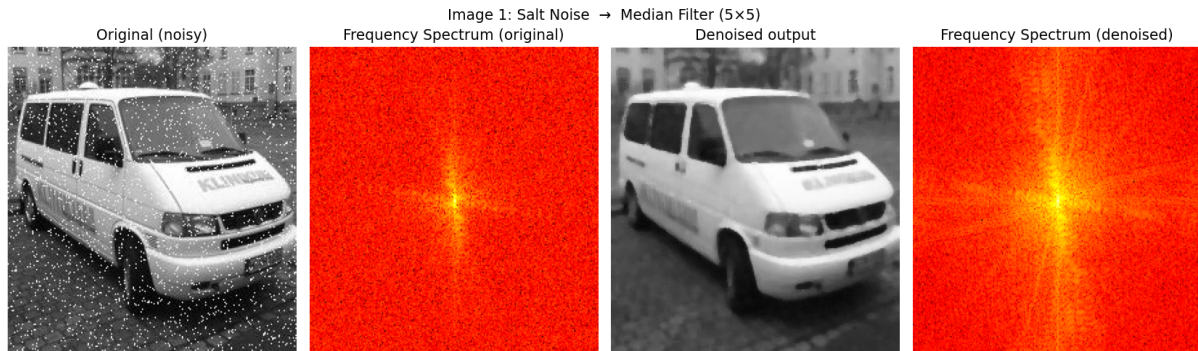


Figure 1. Image 1: original | Fourier spectrum | denoised output | denoised spectrum. SNR improvement: +10.6 dB.

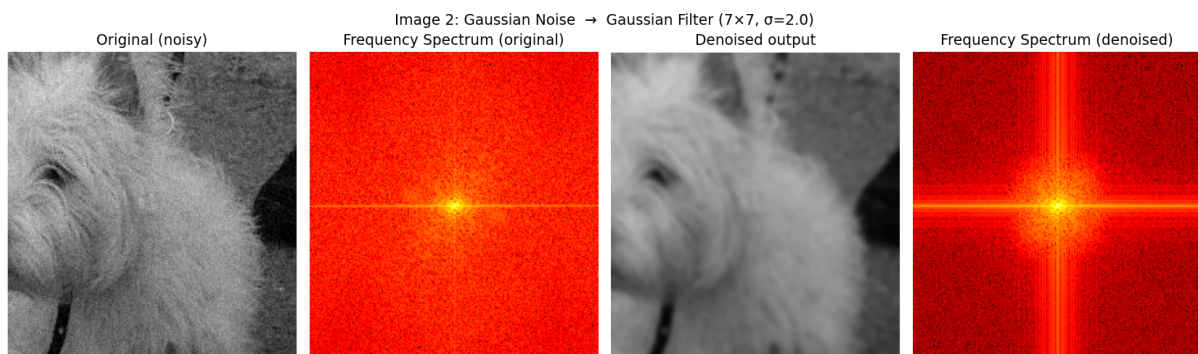


Figure 2. Image 2: original | Fourier spectrum | denoised output | denoised spectrum. SNR improvement: +20.6 dB.

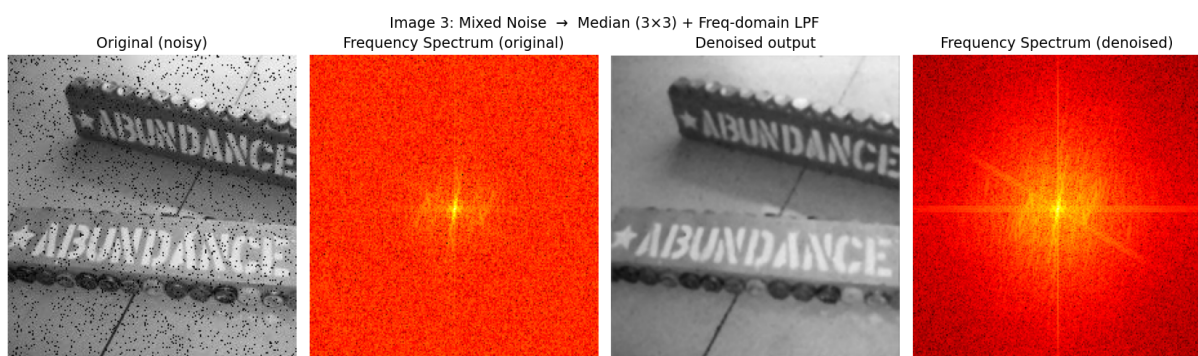


Figure 3. Image 3: original | Fourier spectrum | denoised output | denoised spectrum. SNR improvement: +13.0 dB.

Image 1: The 5×5 median filter cleanly eliminates salt noise (+10.6 dB SNR), preserving sharp vehicle edges and cobblestone texture. Slight softening occurs only in dense noise clusters.

Image 2: The Gaussian filter reduces high-frequency noise energy (+20.6 dB SNR). Some blurring of fine fur detail is expected; a larger kernel trades sharpness for noise suppression. The denoised

spectrum confirms high-frequency attenuation.

Image 3: The two-stage approach achieves +13.0 dB SNR, revealing clear text and decorative elements. The Fourier spectrum comparison confirms removal of periodic high-frequency components. Good contrast and legibility of "ABUNDANCE" is preserved.

4 Summary

The primary challenge was implementing efficient image-processing primitives without library support. The solution uses *numpy.lib.stride_tricks.sliding_window_view* for fully vectorised window operations, avoiding slow Python loops over 224×224 pixels.

Key knowledge gained: (1) Different noise types require fundamentally different filters: impulse noise → order-statistics (median); Gaussian noise → linear Gaussian averaging; periodic noise → frequency-domain suppression. (2) Fourier spectrum analysis provides a diagnostic tool to distinguish noise types. (3) A two-stage spatial + frequency-domain pipeline handles mixed noise more effectively than any single filter. (4) Filter parameter selection (kernel size, σ , D) involves a sharpness–noise trade-off that must be tuned based on the noise characteristics.